

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: CÁC HỆ THỐNG THÔNG MINH
DỰ ÁN AIRCARE
DỰ ĐOÁN CHẤT LƯỢNG KHÔNG KHÍ AQI (TỪ PM2.5) TRONG 1, 3, 6 GIỜ
TIẾP THEO VÀ CẢNH BÁO SỨC KHỎE CÁ NHÂN

Giảng viên hướng dẫn: TS. Mai Xuân Tráng

Lớp tín chỉ: CSE702115-1-1-25(N01)

Nhóm 8 – Lớp: KTPM(EL_2)

- | | |
|-----------------------------|----------------------|
| 1. Lê Thị Kiều Trang | MSV: 23010502 |
| 2. Quách Hữu Nam | MSV: 23012358 |
| 3. Nguyễn Minh Sang | MSV: 23010888 |
| 4. Nguyễn Đức Trường | MSV: 23010767 |

Hà Nội, tháng 11 năm 2025

THÀNH VIÊN NHÓM 8

STT	Họ và tên	Mã sinh viên	Tên tài khoản github	Nội dung công việc
1	Lê Thị Kiều Trang	23010502	agnessktt	- Làm tài liệu - Lập trình chương trình
2	Quách Hữu Nam	23012358	NamNamNN	- Thu thập thông tin - Lập trình chương trình
3	Nguyễn Minh Sang	23010888	minhsangcoder	- Thu thập thông tin - Lập trình chương trình
4	Nguyễn Đức Trường	23010767	T-Hades02	- Thu thập thông tin - Lập trình chương trình

MỤC LỤC

1. Giới thiệu	5
1.1 Bối cảnh	5
1.2 Lý do chọn đề tài	6
1.3 Ý nghĩa của dự án.....	6
1.4. Kết nối với xu hướng AI Ngữ nghĩa (Semantic AI)	7
2. Kiến trúc chi tiết	7
2.1. Tổng quan	7
2.2. Cấu trúc thư mục.....	7
2.3. Luồng hoạt động (Workflow)	8
2.4. Công nghệ sử dụng	9
2.5. Kiến trúc lớp (Layered Architecture)	10
3. Dữ liệu (Data)	11
3.1 Nguồn dữ liệu.....	11
3.2. Mẫu dữ liệu thực tế.....	12
3.3. Các trường dữ liệu.....	13
3.4. Quy trình thu thập dữ liệu	14
3.5. Tần suất và phạm vi thu thập.....	15
3.6. Đặc điểm dữ liệu	15
3.7. Mục tiêu sử dụng dữ liệu	15
4. Thuật toán & mô hình huấn luyện.....	15
4.1. Tổng quan	15
4.2. Thuật toán LightGBM (Light Gradient Boosting Machine)	15
4.3. Cấu trúc mô hình huấn luyện.....	17
4.5 Quy đổi PM2.5 sang AQI	24
5. Giao diện AIRCARE	24
5.1 Triển khai giao diện người dùng (UI) bằng Streamlit	24
5.2. Chức năng chính.....	25
5.3. Hiển thị thông tin cụ thể tại thời điểm hiện tại và đưa ra cảnh báo.....	26
5.4. Dự đoán AQI trong 1, 3, 6 giờ tới.....	26
5.5. Cảnh báo khi người dùng nhập thông tin bệnh lý.....	27
6. Hướng phát triển	28

TÀI LIỆU THAM KHẢO.....	29
--------------------------------	-----------

DỰ ÁN AIRCARE

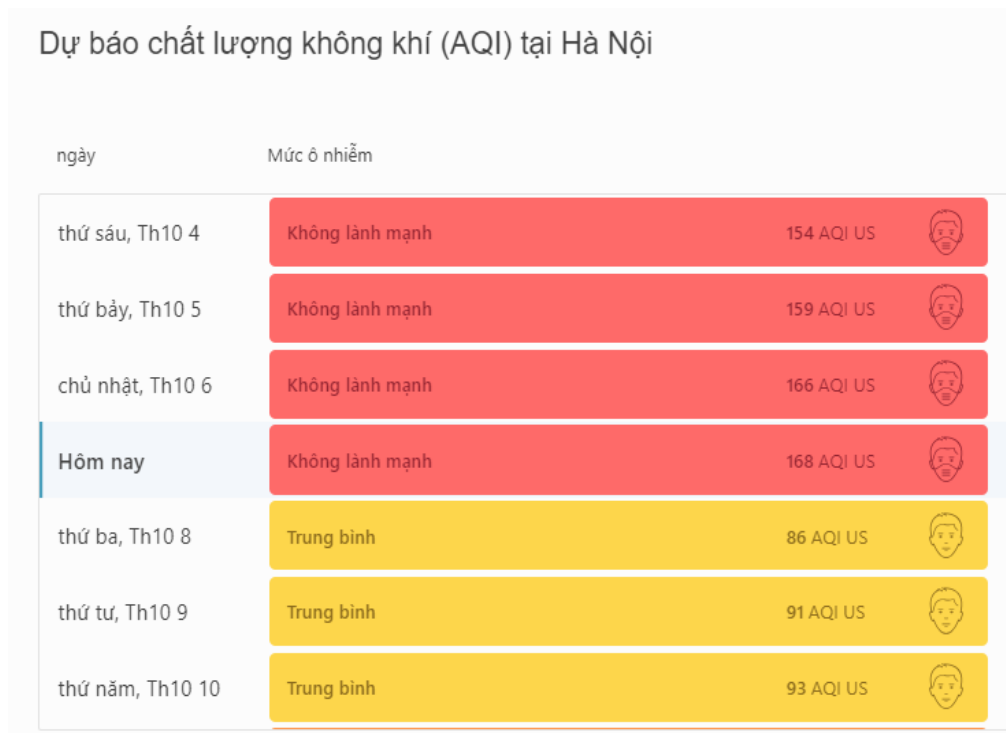
DỰ ĐOÁN CHẤT LƯỢNG KHÔNG KHÍ AQI (TỪ PM2.5) TRONG 1, 3, 6 GIỜ TIẾP THEO VÀ CẢNH BÁO SỨC KHỎE CÁ NHÂN

1. Giới thiệu

1.1 Bối cảnh

Trong những năm gần đây, vấn đề ô nhiễm không khí đã trở thành một trong những thách thức môi trường lớn nhất trên toàn cầu. Theo báo cáo của Tổ chức Y tế Thế giới (WHO, 2024), ô nhiễm không khí là nguyên nhân khiến hơn 7 triệu người tử vong mỗi năm, chủ yếu do các bệnh về tim mạch, hô hấp và ung thư phổi.

Việt Nam, đặc biệt là các đô thị lớn như Hà Nội thường xuyên nằm trong nhóm các thành phố có chỉ số chất lượng không khí (AQI) ở mức “Kém” hoặc “Có hại cho sức khỏe”, đặc biệt vào mùa đông hoặc các thời điểm giao mùa.



Hình 1.1 Dự báo chất lượng không khí (AQI) tại Hà Nội

Nguồn: AQI

Trong 4 ngày, từ 4/10 đến 7/10, Hà Nội liên tục ô nhiễm không khí ở mức nghiêm trọng. Song song với sự phát triển của công nghệ Internet of Things (IoT) và Trí tuệ nhân tạo (AI), các hệ thống cảm biến và nền tảng dữ liệu mở như OpenWeatherMap, AirVisual, Aqicn.org đã tạo điều kiện thuận lợi để người dùng có thể truy cập và theo dõi dữ liệu không khí theo thời gian thực. Tuy nhiên, phần lớn các hệ

thống hiện nay chỉ dừng lại ở mức hiển thị dữ liệu hiện tại, chưa có khả năng dự đoán xu hướng ô nhiễm trong tương lai hay cá nhân hóa cảnh báo theo từng đối tượng người dùng.

Trong bối cảnh đó, Dự án AIRCARE được xây dựng như một ứng dụng Machine Learning nhằm dự báo và cảnh báo chất lượng không khí (AQI) theo thời gian thực. Hệ thống tập trung vào việc dự đoán nồng độ PM2.5 — loại hạt bụi mịn nguy hiểm nhất và là thành phần chính gây ô nhiễm không khí — trong ngắn hạn, bao gồm các khoảng 1 giờ, 3 giờ và 6 giờ tiếp theo. Từ giá trị PM2.5 dự báo, hệ thống tự động quy đổi sang chỉ số AQI tương ứng. Mô hình sử dụng các dữ liệu môi trường thực tế như: pm2_5, pm10, no2, co, o3, so2, temp, humidity để đảm bảo độ chính xác cao. Bên cạnh khả năng dự báo, AIRCARE còn tích hợp một chatbot thông minh giúp cá nhân hóa cảnh báo sức khỏe dựa trên độ tuổi, thể trạng và các bệnh lý của từng người dùng. Nhờ đó, người dùng có thể hiểu rõ mức độ nguy hại của chất lượng không khí hiện tại và nhận được khuyến nghị phù hợp, chẳng hạn như có nên ra ngoài, có cần đeo khẩu trang hay hạn chế vận động. Hệ thống không chỉ hỗ trợ đưa ra quyết định hằng ngày mà còn góp phần nâng cao nhận thức cộng đồng về sức khỏe môi trường.

1.2 Lý do chọn đề tài

Lựa chọn đề tài xuất phát từ thực tế ô nhiễm không khí tại Việt Nam đang ngày càng nghiêm trọng, đòi hỏi những công cụ cảnh báo thông minh, trực quan và dễ sử dụng để hỗ trợ người dân bảo vệ sức khỏe. Hiện nay, phần lớn các ứng dụng chỉ cung cấp chỉ số AQI tại thời điểm hiện tại mà thiếu các giải pháp dự đoán ngắn hạn, khiến người dùng khó có sự chuẩn bị và phòng ngừa kịp thời. Trong bối cảnh công nghệ Học máy và Xử lý ngôn ngữ tự nhiên phát triển mạnh mẽ, việc xây dựng một hệ thống vừa có khả năng dự đoán chính xác, vừa tương tác tự nhiên với chi phí thấp trở nên khả thi. Sự kết hợp giữa mô hình dự đoán PM2.5 (LightGBM) và mô hình hiểu ngôn ngữ tự nhiên (SentenceTransformer) tạo nên một hệ thống toàn diện, giúp người dùng không chỉ nhận được dự báo đáng tin cậy mà còn có thể đặt câu hỏi và nhận phản hồi theo ngôn ngữ tự nhiên. Bên cạnh giá trị thực tiễn trước mắt, dự án còn có tiềm năng mở rộng trong bối cảnh đô thị thông minh (Smart City), tích hợp IoT và bản đồ chất lượng không khí trong tương lai.

1.3 Ý nghĩa của dự án

Về mặt công nghệ, AIRCARE được xây dựng như một hệ thống AI lai (Hybrid AI), kết hợp giữa mô hình Machine Learning — sử dụng LightGBM để dự đoán chuỗi thời gian PM2.5 — và mô hình NLP dựa trên SentenceTransformer để nhận diện ý định người dùng, giúp chatbot phản hồi chính xác và tự nhiên. Toàn bộ quy trình xử lý dữ liệu được vận hành theo một pipeline hoàn chỉnh, bao gồm các bước thu thập, làm sạch, huấn luyện và triển khai, đảm bảo tính tự động và ổn định của hệ thống. Hơn nữa, AIRCARE hỗ trợ khả năng suy luận thời gian thực (real-time inference), cho phép dự báo và đưa ra cảnh báo tức thì cho người dùng.

Về mặt xã hội, hệ thống mang lại nhiều lợi ích thiết thực như hỗ trợ người dân theo dõi và dự đoán mức độ ô nhiễm không khí một cách dễ dàng, đặc biệt hữu ích đối với những người có bệnh lý hô hấp như hen suyễn, viêm phổi mạn tính hoặc người cao tuổi — những nhóm đối tượng nhạy cảm với sự thay đổi của chất lượng không khí. Bên cạnh đó, AIRCARE còn góp phần nâng cao nhận thức cộng đồng về tác động của ô nhiễm môi trường đến sức khỏe, từ đó thúc đẩy thói quen sống an toàn và các hành vi sinh hoạt lành mạnh hơn.

1.4. Kết nối với xu hướng AI Ngữ nghĩa (Semantic AI)

Giai đoạn 2024–2025 chứng kiến sự bùng nổ mạnh mẽ của các mô hình ngôn ngữ trong việc hiểu và tương tác tự nhiên với con người, kéo theo sự xuất hiện ngày càng phổ biến của các hệ thống AI trợ lý môi trường tại nhiều quốc gia. Dự án AIRCARE hòa nhập vào xu thế này bằng cách ứng dụng các mô hình nhúng ngữ nghĩa như SentenceTransformer để hiểu ý định của người dùng thông qua ngôn ngữ tự nhiên, thay vì các tương tác dạng nút bấm truyền thống. Bên cạnh đó, hệ thống được kết nối trực tiếp với dữ liệu cảm biến môi trường thực tế, giúp các phản hồi và cảnh báo — vốn được thiết kế có cấu trúc rõ ràng — trở nên chính xác và phù hợp hơn với tình trạng không khí tại thời điểm truy vấn. AIRCARE không chỉ cung cấp thông tin đơn thuần mà còn hướng tới một trải nghiệm tương tác cá nhân hóa, đem lại cảm giác như người dùng đang trò chuyện với một trợ lý môi trường thông minh thay vì chỉ quan sát một dashboard dữ liệu tĩnh.

2. Kiến trúc chi tiết

2.1. Tổng quan

Dự án AIRCARE — Dự đoán AQI (từ PM2.5) được thiết kế theo mô hình modular architecture, gồm 4 lớp chính:

- + Thu thập dữ liệu (Data Collection) – lấy dữ liệu thời tiết & không khí thực từ API OpenWeather.
- + Tiền xử lý & Huấn luyện mô hình (Model Training) – làm sạch dữ liệu, huấn luyện và lưu model dự đoán.
- + Triển khai API & Chatbot (Application Layer) – cung cấp giao diện giao tiếp và dự đoán theo thời gian thực.
- + Lưu trữ & kiểm thử (Testing & Storage) – quản lý dữ liệu, mô hình, và kiểm thử các endpoint.

2.2. Cấu trúc thư mục

AIRCARE/

	— .venv312/	#Môi trường ảo Python (tự động tạo khi dùng venv)
	— scripts/	#Chứa file kích hoạt môi trường (activate)
	— lib/, include/, etc.	# Các thư viện Python cài trong venv
	— data/	#Thư mục dữ liệu
	— logs/	#File nhật ký tự động
	— collector.log	#File log khi thu thập dữ liệu
	— raw/	#Dữ liệu gốc (chưa xử lý)
	— air_data.csv	#File dữ liệu AQI gốc (các thông số môi trường)

	└─ models/	#Thư mục lưu các mô hình đã huấn luyện
	└─ pm2_5_model_1h.pkl	#Mô hình dự đoán PM2.5 1 giờ tới
	└─ pm2_5_model_3h.pkl	#Mô hình dự đoán PM2.5 3 giờ tới
	└─ pm2_5_model_6h.pkl	#Mô hình dự đoán PM2.5 6 giờ tới
	└─ pm2_5_features_1h.pkl	#Danh sách đặc trưng tương ứng với model 1h
	└─ pm2_5_features_3h.pkl	#Danh sách đặc trưng tương ứng với model 3h
	└─ pm2_5_features_6h.pkl	#Danh sách đặc trưng tương ứng với model 6h
	└─ src/	#Thư mục mã nguồn chính của dự án
	└─ data/	#Chứa các script thu thập & xử lý dữ liệu
	└─ .env	#File lưu biến môi trường (API key, URL, v.v.)
	└─ collector.py	#Script tự động thu thập dữ liệu AQI từ web
	└─ models/	#Chứa các script huấn luyện mô hình
	└─ train_model.py	#Script huấn luyện và lưu các mô hình vào /models
	└─ run/	#Thư mục chạy ứng dụng chính
	└─ app.py	#File Streamlit chính — giao diện chatbot AQI
	└─ .gitignore	#Khai báo các file/thư mục không đẩy lên GitHub (.venv312)
	└─ README.md	#Mô tả tổng quan dự án, cách chạy, cấu trúc, tính năng
	└─ requirements.txt	#Danh sách thư viện cần

2.3. Luồng hoạt động (Workflow)

Bước 1 – Thu thập dữ liệu

Người dùng chạy file collector.py → hệ thống gọi API OpenWeather → thu thập thông tin thời tiết và các chỉ số không khí (quan trọng nhất là PM2.5) → lưu vào data/raw/air_data.csv.

Bước 2 – Tiền xử lý và huấn luyện mô hình

Chạy file train_model.py → dữ liệu được làm sạch, tạo đặc trưng (features) → huấn luyện mô hình LightGBM (Light Gradient Boosting Machine) → xuất ra 3 model .pkl (dự đoán nồng độ PM2.5 sau 1h, 3h, 6h).

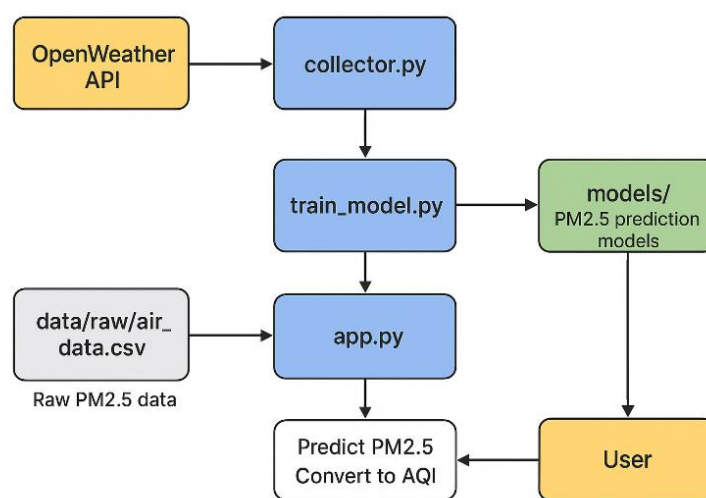
Bước 3 – Triển khai ứng dụng AI

+ Tải các mô hình pm2_5_model_...pkl đã huấn luyện.

- + Nhận đầu vào từ người dùng (ví dụ: "dự đoán không khí").
- + Sử dụng mô hình để dự đoán giá trị PM2.5 (ví dụ: 45.7 $\mu\text{g}/\text{m}^3$).
- + Ứng dụng quy đổi giá trị PM2.5 này sang chỉ số AQI (ví dụ: 125).
- + Trả về kết quả AQI đã quy đổi, cùng lời khuyên sức khỏe phù hợp.

Bước 4 – Hiển thị kết quả và cảnh báo

- + Biểu đồ dự đoán AQI (đã quy đổi).
- + Cảnh báo sức khỏe theo từng mức AQI.
- + Chatbot hỗ trợ hỏi đáp bằng ngôn ngữ tự nhiên (dựa trên việc phát hiện ý định).



Hình 2.1 Luồng hoạt động (Workflow)

2.4. Công nghệ sử dụng

Hệ thống AIRCARE được xây dựng dựa trên các công nghệ và thư viện hiện đại:

- + Python: Ngôn ngữ lập trình chính.
- + Pandas và NumPy: Dùng để xử lý, làm sạch, và phân tích dữ liệu.
- + Scikit-learn và LightGBM: Hỗ trợ xây dựng và huấn luyện mô hình dự đoán PM2.5.
- + Joblib: Dùng để lưu trữ và tải lại các mô hình huấn luyện (.pkl).
- + OpenWeatherMap API: Cung cấp dữ liệu thời tiết và chất lượng không khí theo thời gian thực, được sử dụng trong quá trình thu thập dữ liệu định kỳ 2 phút/lần.
- + AI Ngữ nghĩa (NLP): Sử dụng SentenceTransformer (một mô hình AI Ngữ nghĩa) để hiểu ý định (intent detection) của người dùng qua ngôn ngữ tự nhiên, giúp chatbot chọn ra phản hồi phù hợp.
- + Streamlit: Thư viện Python để xây dựng và triển khai giao diện người dùng (UI) dạng chatbot một cách nhanh chóng.

2.5. Kiến trúc lớp (Layered Architecture)

Hệ thống AIRCARE được thiết kế theo mô hình kiến trúc nhiều lớp (Layered Architecture), giúp phân tách rõ ràng các chức năng, tăng tính mở rộng, dễ bảo trì và thuận tiện khi triển khai.

Cụ thể, hệ thống bao gồm 4 lớp chính như sau:

* Lớp Thu thập dữ liệu (Data Collection Layer)

- Chức năng: Tự động thu thập dữ liệu không khí thực tế (bao gồm PM2.5, PM10, nhiệt độ, v.v.) từ web API định kỳ (mỗi 2 phút).

- Thực hiện tại: collector.py

- Nhiệm vụ:

- + Gọi API OpenWeather.

- + Chuẩn hóa, làm sạch dữ liệu đầu vào.

- + Lưu dữ liệu vào tệp data/raw/air_data.csv.

* Lớp Xử lý và huấn luyện mô hình (Processing & Model Layer)

- + Chức năng: Xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình dự đoán PM2.5.

- + Thực hiện tại: train_model.py

- + Thuật toán sử dụng: LightGBM Regressor (Gradient Boosting Decision Tree).

- + Nhiệm vụ:

- + Làm sạch dữ liệu và tạo các đặc trưng (feature engineering).

- + Huấn luyện 3 mô hình riêng biệt cho dự báo PM2.5 sau 1h, 3h và 6h.

- + Lưu các mô hình đã huấn luyện vào thư mục models/ dưới dạng .pkl.

* Lớp Ứng dụng và Giao diện người dùng (Application/UI Layer)

- Chức năng: Hiển thị thông tin AQI hiện tại, dự đoán tương lai và hỗ trợ trò chuyện với người dùng.

- Thực hiện tại: app.py (xây dựng bằng Streamlit).

- Nhiệm vụ:

- + Tải mô hình dự đoán PM2.5 đã huấn luyện.

- + Quy đổi các giá trị PM2.5 (hiện tại và dự đoán) sang chỉ số AQI để hiển thị.

- + Giao diện chatbot hỗ trợ người dùng hỏi – đáp bằng ngôn ngữ tự nhiên (dựa trên SentenceTransformer).

- + Hiển thị biểu đồ, cảnh báo sức khỏe, và khuyến nghị theo từng mức AQI.

* Lớp Lưu trữ và Quản lý dữ liệu (Data Storage Layer)

- Chức năng: Lưu trữ dữ liệu và mô hình để phục vụ cho cả quá trình huấn luyện và dự đoán.

- Thực hiện tại:

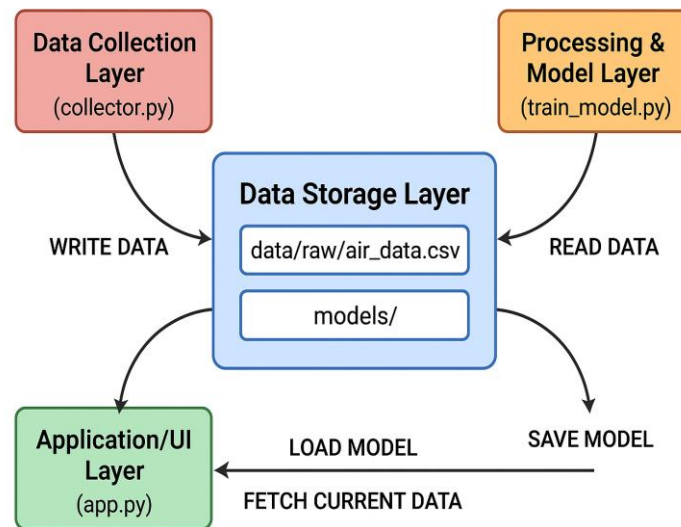
+ data/raw/: chứa dữ liệu gốc thu thập được.

+ models/: chứa các mô hình LightGBM (dự đoán PM2.5) đã huấn luyện và tệp danh sách đặc trưng.

- Nhiệm vụ:

+ Đảm bảo toàn vẹn dữ liệu.

+ Cung cấp dữ liệu cho lớp dự đoán và giao diện.



Hình 2.2 Kiến trúc lớp (Layered Architecture)

3. Dữ liệu (Data)

3.1 Nguồn dữ liệu

Dữ liệu được thu thập từ OpenWeatherMap API, một nền tảng cung cấp thông tin thời tiết và chất lượng không khí theo thời gian thực.

Mỗi lần người dùng chạy chương trình collector.py, hệ thống sẽ:

- Gửi yêu cầu đến API để xác định tọa độ thành phố (như Hà Nội).
- Gọi API Air Pollution và Weather để nhận các chỉ số môi trường.
- Gộp dữ liệu này và lưu vào file CSV cục bộ.

Dữ liệu được lưu tại: data/raw/air_data.csv

3.2. Mẫu dữ liệu thực tế



Hình 3.1 Dự báo thời tiết VTV. (2025, 16 November). Không khí Hà Nội sáng nay tiếp tục ô nhiễm nặng [Facebook post]. TTXVN.

data > raw > air_data.csv > data	
1	timestamp,pm2_5,pm10,no2,co,o3,so2,temp,humidity,aqi
784	2025-11-16 00:14:22+0700,93.2,104.34,5.52,434.98,22.37,1.79,20.98,75,5
785	2025-11-16 09:46:18+0700,78.04,85.09,5.51,367.55,42.27,3.41,22.98,71,5
786	2025-11-16 09:48:20+0700,78.04,85.09,5.51,367.55,42.27,3.41,22.98,71,5
787	2025-11-16 09:50:22+0700,78.04,85.09,5.51,367.55,42.27,3.41,22.98,71,5
788	2025-11-16 09:52:23+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
789	2025-11-16 09:54:25+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
790	2025-11-16 09:56:27+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
791	2025-11-16 09:58:28+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
792	2025-11-16 10:00:30+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
793	2025-11-16 10:02:32+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
794	2025-11-16 10:04:34+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
795	2025-11-16 10:06:35+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
796	2025-11-16 10:08:37+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
797	2025-11-16 10:10:39+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,71,5
798	2025-11-16 10:12:40+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.97,70,5
799	2025-11-16 10:14:42+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.97,70,5
800	2025-11-16 10:16:44+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.97,70,5
801	2025-11-16 10:18:45+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.97,70,5
802	2025-11-16 10:20:47+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.97,70,5
803	2025-11-16 10:22:49+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,70,5
804	2025-11-16 10:24:50+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,70,5
805	2025-11-16 10:26:52+0700,78.04,85.09,5.51,367.55,42.27,3.41,23.98,70,5

Hình 3.2 Dữ liệu thực tế từ OpenWeatherMap API

Dữ liệu được thu thập từ OpenWeatherMap API và lưu trong tệp data/raw/air_data.csv, với chu kỳ cập nhật khoảng 2 phút/lần. Mỗi dòng dữ liệu đại diện cho một lần ghi nhận thông số môi trường tại Hà Nội trong sáng ngày 16/11/2025.

Kết quả thu thập cho thấy nồng độ PM2.5 duy trì ổn định quanh mức xấp xỉ 78 $\mu\text{g}/\text{m}^3$ trong suốt thời gian quan sát. Khi quy đổi theo thang đo của US EPA, mức PM2.5 này tương ứng với AQI ≈ 155 , thuộc nhóm “Có hại cho sức khỏe” (Unhealthy for Sensitive Groups). Giá trị này hoàn toàn trùng khớp với cảnh báo do Dự báo Thời tiết VTV công bố cùng ngày, trong đó ghi nhận AQI tại các trạm đo ở Hà Nội dao động từ 150 đến 172.

Sự tương đồng giữa dữ liệu thu thập tự động và số liệu cảnh báo từ cơ quan truyền thông chính thống chứng minh pipeline thu thập dữ liệu của hệ thống hoạt động chính xác và đáng tin cậy. Đồng thời, kết quả này xác nhận mức độ ô nhiễm không khí tại Hà Nội đã vượt ngưỡng khuyến nghị an toàn của WHO.

Việc nồng độ PM2.5 duy trì liên tục ở mức cao, không biến động đột ngột, cho thấy đây là ô nhiễm nền chứ không phải hiện tượng ngắn hạn, phản ánh đặc trưng thời tiết nghịch nhiệt và tích tụ bụi trong đô thị. Đặc điểm này giúp mô hình LightGBM học được quy luật biến động của các chỉ số môi trường theo thời gian, từ đó cho phép hệ thống AIRCARE dự đoán chất lượng không khí trong 1, 3 và 6 giờ tiếp theo với độ tin cậy cao, đáp ứng mục tiêu cảnh báo sớm và bảo vệ sức khỏe cộng đồng.

Trong dữ liệu thực tế, trường aqi = 5 không phải là AQI theo thang 0–500, mà là chỉ số phân loại mức độ ô nhiễm của OpenWeatherMap, trong đó 5 là mức “Very Poor – Rất có hại cho sức khỏe”. Vì vậy, dự án không dùng trực tiếp cột này để huấn luyện mà thay vào đó dự đoán PM2.5 và quy đổi sang AQI chuẩn US EPA để cảnh báo chi tiết hơn.

3.3. Các trường dữ liệu

Trường	Kiểu dữ liệu	Đơn vị	Mô tả
timestamp	datetime	YYYY-MM-DD HH:MM:SS	Thời điểm ghi nhận dữ liệu.
pm2_5	float	$\mu\text{g}/\text{m}^3$	Nồng độ bụi mịn có kích thước nhỏ hơn 2.5 micromet. Gây hại trực tiếp cho phổi, tim và mạch máu. Đây là biến mục tiêu chính mà mô hình Học máy sẽ dự đoán.
pm10	float	$\mu\text{g}/\text{m}^3$	Nồng độ bụi thô có kích thước nhỏ hơn 10 micromet.
no2	float	$\mu\text{g}/\text{m}^3$	Nồng độ Nitơ Dioxid – khí thải từ phương tiện và nhà máy.
co	float	$\mu\text{g}/\text{m}^3$	Nồng độ Carbon Monoxit – khí độc gây ngạt, chủ yếu do đốt cháy nhiên liệu.
o3	float	$\mu\text{g}/\text{m}^3$	Nồng độ Ozon tầng thấp – có thể gây kích ứng hô hấp.
so2	float	$\mu\text{g}/\text{m}^3$	Nồng độ Lưu huỳnh Dioxid – khí gây viêm phổi và đau họng khi hít lâu.

temp	float	°C	Nhiệt độ không khí hiện tại.
humidity	integer	%	Độ ẩm tương đối trong không khí.
aqi	integer	chuẩn hóa từ thang 1–5)	Chỉ số chất lượng không khí. Giá trị càng cao → không khí càng ô nhiễm.

3.4. Quy trình thu thập dữ liệu

Quá trình thu thập được thực hiện theo chu trình tự động như sau:

- Khởi chạy lệnh: python src/data/collector.py
- Hệ thống gửi yêu cầu đến API

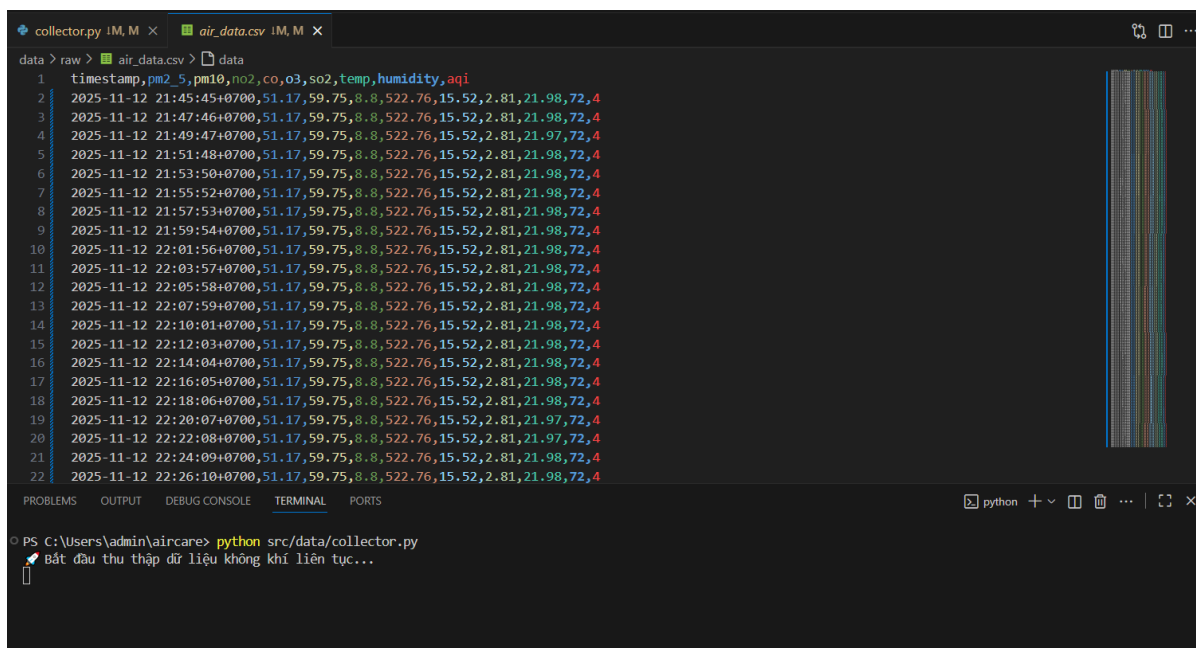
```

40 # ----- Lấy tọa độ -----
41 geo_url = f"http://api.openweathermap.org/geo/1.0/direct?q={city}&limit=1&appid={API_KEY}"
42 geo_resp = requests.get(geo_url, timeout=10).json()
43
44 if not isinstance(geo_resp, list) or len(geo_resp) == 0:
45     raise ValueError(f"City '{city}' not found.")
46
47 lat, lon = geo_resp[0]["lat"], geo_resp[0]["lon"]
48
49 # ----- Lấy dữ liệu không khí -----
50 air_url = f"http://api.openweathermap.org/data/2.5/air_pollution?lat={lat}&lon={lon}&appid={API_KEY}"
51 air_data = requests.get(air_url, timeout=10).json()
52
53 # ----- Lấy dữ liệu thời tiết -----
54 weather_url = f"https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API_KEY}&units=metric"
55 weather_data = requests.get(weather_url, timeout=10).json()
56

```

Hình 3.3 Gửi các yêu cầu API đến dịch vụ OpenWeatherMap để thu thập dữ liệu

- Nhận phản hồi JSON, trích xuất các chỉ số cần thiết.
- Tạo dòng dữ liệu và thêm vào file CSV data/raw/air_data.csv.



Hình 3.4 Xử lý dữ liệu và thêm vào cơ sở dữ liệu (CSDL)

3.5. Tần suất và phạm vi thu thập

- Khu vực: Thành phố Hà Nội (mặc định), có thể mở rộng thêm nhiều thành phố khác bằng cách thay đổi biến DEFAULT_CITY trong .env.
- Tần suất gửi liệu cập nhật: Thời gian thực – thông qua OpenWeather API (độ trễ trung bình 2 phút).

3.6. Đặc điểm dữ liệu

- Kích thước mẫu ban đầu: Mỗi lần thu thập tạo ra một dòng dữ liệu mới, có thể tích lũy thành bộ dữ liệu lớn theo thời gian.
- Độ tin cậy: Nguồn dữ liệu chính thống, được cập nhật liên tục bởi OpenWeatherMap.
- Độ nhiễu: Thấp, tuy nhiên có thể phát sinh giá trị thiếu (NaN) khi API trả về lỗi hoặc mạng gián đoạn.
- Định dạng: CSV.

3.7. Mục tiêu sử dụng dữ liệu

Dữ liệu sau khi xử lý được dùng để:

- Huấn luyện mô hình dự đoán nồng độ PM2.5 theo thời gian (dự báo 1h, 3h, 6h).
- Cung cấp dữ liệu (PM2.5) cho ứng dụng app.py để quy đổi sang chỉ số AQI và hiển thị cho người dùng.
- Phát hiện ý định (intent detection) cho chatbot thông minh (dựa trên SentenceTransformer) để cảnh báo người dùng theo ngữ cảnh thực tế.

4. Thuật toán & mô hình huấn luyện

4.1. Tổng quan

Trong dự án AIRCARE – Chatbot Dự đoán AQI, nhóm sử dụng hai thành phần AI chính:

- Thuật toán học máy (Machine Learning): Sử dụng LightGBM để dự đoán nồng độ PM2.5 (một giá trị liên tục, đơn vị $\mu\text{g}/\text{m}^3$) trong các khung thời gian tương lai (1h, 3h, 6h).
- Mô hình hiểu ngôn ngữ (NLP / AI Ngữ nghĩa): Sử dụng SentenceTransformer để phát hiện ý định (intent detection) từ câu chat của người dùng (ví dụ: "có nên ra ngoài không?").

Đây là bài toán hồi quy (regression), vì mô hình học cách dự đoán giá trị pm2_5 (một chỉ số liên tục). Sau đó, ứng dụng (app.py) sẽ lấy kết quả PM2.5 này và quy đổi sang chỉ số AQI (1-5) để hiển thị cho người dùng.

4.2. Thuật toán LightGBM (Light Gradient Boosting Machine)

❖ Khái niệm

LightGBM là một thuật toán Boosting dạng cây quyết định (Tree-Based Gradient Boosting), được phát triển bởi Microsoft.

Thuật toán hoạt động bằng cách xây dựng nhiều cây yếu (weak learners) theo chuỗi, trong đó mỗi cây mới học từ sai số (residual) của các cây trước đó.

❖ Cách mô hình LightGBM hoạt động trong hệ thống AirCare

Trong dự án AirCare, thuật toán LightGBM (Light Gradient Boosting Machine) được sử dụng để dự báo nồng độ PM2.5 trong tương lai (1 giờ, 3 giờ, 6 giờ). Đây là mô hình học máy mạnh mẽ, phù hợp cho bài toán dự báo theo thời gian nhờ khả năng xử lý dữ liệu phi tuyến và tốc độ huấn luyện nhanh.

- Nguyên lý cốt lõi của LightGBM

LightGBM là một thuật toán Gradient Boosting Decision Tree (GBDT), nghĩa là:

- + Mô hình gồm hàng trăm cây quyết định nhỏ, không phải chỉ một cây.
- + Các cây được sinh ra tuần tự, mỗi cây mới sẽ cố gắng giảm lỗi mà các cây trước tạo ra.
- + Tổng hợp tất cả các cây lại → mô hình cuối cùng mạnh hơn nhiều.

- Cơ chế hoạt động từng bước:

Bước 1 – Huấn luyện cây đầu tiên

Mô hình tạo ra một cây quyết định đơn giản dự đoán PM2.5 dựa trên các đặc trưng như:

- + temp (nhiệt độ)
- + humidity (độ ẩm)
- + Khí NO₂, CO, SO₂, O₃
- + PM2.5 hiện tại.
- + Giờ trong ngày (datetime)

v.v.

Bước 2 – Tính “phần lỗi”

Sau cây đầu tiên, mô hình xem những giá trị đã dự báo sai bao nhiêu → tạo ra “residual error”.

Bước 3 – Sinh ra cây tiếp theo để sửa lỗi

Cây thứ hai học từ chính phần lỗi này.

Cây thứ ba sửa lỗi cây 1+2.

...

Cứ như vậy lặp lại khoảng 100–500 cây.

Bước 4 – Tổng hợp kết quả

Dự đoán cuối cùng = tổng các dự đoán từ tất cả các cây → giống như “lấy ý kiến nhiều chuyên gia”.

- LightGBM đã được sử dụng cụ thể trong hệ thống như sau

Input (Đầu vào): Từ dữ liệu collector.py thu thập:

- PM2.5 hiện tại
- PM10
- NO₂
- CO

- O_3
- SO_2
- nhiệt độ
- độ ẩm
- timestamp

Output (Đầu ra): 3 mô hình dự đoán PM2.5 trong

- 1 giờ → pm2_5_model_1h.pkl
- 3 giờ → pm2_5_model_3h.pkl
- 6 giờ → pm2_5_model_6h.pkl

- Cách mô hình hoạt động khi dự đoán

Khi chạy app.py:

- + Tải mô hình đã huấn luyện (.pkl)
- + Lấy dữ liệu PM2.5 hiện tại
- + Mô hình tính toán dự báo PM2.5 tương lai
- + Áp dụng công thức AQI để chuyển PM2.5 → chỉ số AQI
- + Trả kết quả về giao diện Streamlit
- Hạn chế của LightGBM trong dự án AIRCARE
- + Cần nhiều dữ liệu (> 30 ngày) để tăng R^2
- + Dễ bị nhiễu nếu dữ liệu thời tiết cập nhật không đều

→ LightGBM được lựa chọn cho dự án AirCare vì sở hữu nhiều ưu điểm phù hợp với bài toán dự báo chất lượng không khí. Trước hết, mô hình này có tốc độ huấn luyện rất nhanh và tiêu tốn ít tài nguyên, giúp xử lý hiệu quả ngay cả khi dữ liệu thời gian thực được thu thập liên tục mỗi 2 phút. Bên cạnh đó, LightGBM xử lý tốt các mối quan hệ phi tuyến giữa nhiều thông số môi trường vốn biến động phức tạp, điều mà các mô hình tuyến tính như Linear Regression khó đáp ứng. Một ưu điểm quan trọng khác là LightGBM không yêu cầu chuẩn hóa dữ liệu, cho phép mô hình làm việc trực tiếp với các đặc trưng có thang đo khác nhau như PM2.5, nhiệt độ, độ ẩm hay CO. Cuối cùng, LightGBM hỗ trợ hiệu quả các đặc trưng thời gian như giờ, ngày hoặc xu hướng biến động trước đó — những yếu tố đặc biệt quan trọng trong việc dự báo PM2.5 theo chuỗi thời gian.

4.3. Cấu trúc mô hình huấn luyện

- Mã nguồn huấn luyện được cài đặt trong file: src/models/train_model.py

```
train_model.py IM, M X
src > models > train_model.py
1 import os
2 import pandas as pd
3 import numpy as np
4 import lightgbm as lgb
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import mean_absolute_error, r2_score
7 import joblib
8
9 # =====
10 # 🌐 CẤU HÌNH ĐƯỜNG DẪN VÀ LOGIC THỜI GIAN
11 # =====
12 BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), "../../"))
13 DATA_PATH = os.path.join(BASE_DIR, "data/raw/air_data.csv")
14 MODEL_DIR = os.path.join(BASE_DIR, "models")
15 HORIZONS = [1, 3, 6] # dự báo 1h, 3h, 6h tới
16 os.makedirs(MODEL_DIR, exist_ok=True)
17
18 # Dữ liệu được thu thập mỗi 2 phút (theo file collector.py)
19 INTERVAL_MINUTES = 2
20 ROWS_PER_HOUR = 60 // INTERVAL_MINUTES # 30 hàng = 1 giờ
21
22 # =====
23 # 📁 NẠP DỮ LIỆU
24 # =====
25 if not os.path.exists(DATA_PATH):
26     raise FileNotFoundError(f"❌ Không tìm thấy file dữ liệu tại {DATA_PATH}")
27
28 df = pd.read_csv(DATA_PATH, parse_dates=["timestamp"])
29 df = df.drop_duplicates("timestamp").sort_values("timestamp").reset_index(drop=True)
30 df = df.dropna(subset=["pm2_5"])
31 print(f"✅ Nạp dữ liệu: {len(df)} dòng, {df['timestamp'].min()} → {df['timestamp'].max()}")
32
33 # =====
34 # 🌱 TẠO FEATURE CHO PM2.5
35 # =====
36 def create_features(df, shift_rows):
37     """Tạo feature và target cho từng horizon."""
38     df_feat = df.copy()
```

Hình 4.1 Đọc, làm sạch và chuẩn bị dữ liệu

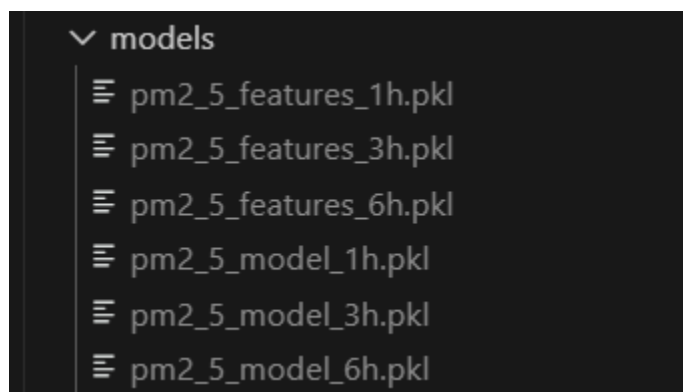
- Mô hình được huấn luyện tự động cho 3 mốc thời gian dự báo, với mục tiêu là dự đoán nồng độ PM2.5:

+ Nồng độ PM2.5 sau 1 giờ

+ Nồng độ PM2.5 sau 3 giờ

+ Nồng độ PM2.5 sau 6 giờ

Mỗi mô hình được lưu lại dạng .pkl trong thư mục models/.



Hình 4.2 Kết quả đầu ra sau khi chạy script huấn luyện mô hình

4.4. Quy trình huấn luyện mô hình

(1) Nạp và xử lý dữ liệu

- Dữ liệu thực tế từ OpenWeatherMap được lưu tại data/raw/air_data.csv.
- Sau khi đọc, dữ liệu được sắp xếp theo thời gian (timestamp), loại bỏ bản ghi trùng lặp và giá trị thiếu (NaN).

```
28 df = pd.read_csv(DATA_PATH, parse_dates=["timestamp"])
29 df = df.drop_duplicates("timestamp").sort_values("timestamp").reset_index(drop=True)
30 df = df.dropna(subset=["pm2_5"])
```

+ parse_dates=["timestamp"]: Chuyển cột thời gian sang kiểu datetime giúp thuận tiện cho việc trích xuất đặc trưng thời gian.

+ drop_duplicates("timestamp"): Xóa các bản ghi trùng thời gian nhằm tránh sai lệch khi huấn luyện.

+ dropna(subset=["pm2_5"]): Loại bỏ dòng bị thiếu giá trị AQI để đảm bảo đầu ra huấn luyện chính xác (vì pm2_5 là biến mục tiêu mà mô hình sẽ dự đoán).

(2) Feature Engineering – Xây dựng đặc trưng

- Tạo độ trễ (lag features) cho các chỉ số môi trường như pm2_5, pm10, no2, co, o3, so2, temp và humidity.

- Thêm đặc trưng thời gian như giờ trong ngày (hour), ngày trong tuần (weekday), tháng (month), và biểu diễn tuần hoàn (sin, cos) của giờ để mô hình học chu kỳ ngày–đêm.

+ Các cột hour, weekday, month thể hiện chu kỳ giờ, ngày, tháng.

+ Các cột hour_sin, hour_cos giúp mô hình hiểu chu kỳ tuần hoàn của ngày – đêm, tránh việc mô hình xem 23h và 0h là hai mốc rời rạc.

```
36 def create_features(df, shift_rows):
37     """Tạo feature và target cho từng horizon."""
38     df_feat = df.copy()
39
40     # Thông tin thời gian
41     ts = df_feat["timestamp"]
42     df_feat["hour"] = ts.dt.hour
43     df_feat["weekday"] = ts.dt.weekday
44     df_feat["month"] = ts.dt.month
45     df_feat["hour_sin"] = np.sin(2 * np.pi * df_feat["hour"] / 24)
46     df_feat["hour_cos"] = np.cos(2 * np.pi * df_feat["hour"] / 24)
47
```

+ Dòng lệnh data[f"{col}_lag{lag}"] = data[col].shift(lag) tạo ra các cột mới biểu diễn giá trị các chỉ số (PM2.5, NO₂, nhiệt độ,...) của 1-3 mốc dữ liệu trước đó (tương đương 2-6 phút trước).

+ Tạo đặc trưng thống kê: pm2_5_rolling3 là trung bình trượt của PM2.5 trong 3 mốc dữ liệu, giúp giảm nhiễu và phản ánh xu hướng ngắn hạn.

```

48     # Lag features cho pm2_5 và các khí khác
49     for col in ["pm2_5", "pm10", "co", "no2", "o3", "so2", "temp", "humidity"]:
50         if col in df_feat.columns:
51             for lag in range(1, 7):
52                 df_feat[f"{col}_lag{lag}"] = df_feat[col].shift(lag)
53
54     # Rolling mean
55     df_feat["pm2_5_rol13"] = df_feat["pm2_5"].rolling(window=3).mean()
56

```

(3) Tạo dữ liệu mục tiêu tương lai

- Để huấn luyện mô hình dự đoán PM2.5 (thay vì AQI) trong tương lai, cột mục tiêu được tạo bằng cách dịch (shift) chỉ số pm2_5 về sau.
- Dữ liệu được thu thập mỗi 2 phút (theo collector.py), nghĩa là 1 giờ có $60 / 2 = 30$ hàng (rows) dữ liệu. Do đó, để dự đoán 1, 3, và 6 giờ tới, chúng ta phải dịch chuyển (shift) tương ứng 30, 90, và 180 hàng.

```

74 def train_for_horizon(horizon_in_hours):
75     shift_rows = horizon_in_hours * ROWS_PER_HOUR
76

```

- + Dòng lệnh này tạo ra biến mục tiêu pm2_5_target (trong hàm create_features), đại diện cho giá trị PM2.5 ($\mu\text{g}/\text{m}^3$) trong tương lai (1, 3 hoặc 6 giờ).
- + Mô hình LightGBM sau đó được huấn luyện để dự đoán giá trị pm2_5_target (thay vì aqi_future) dựa trên các đặc trưng hiện tại và quá khứ.

Nhờ đó, hệ thống có thể dự báo xu hướng biến động PM2.5. Giá trị này sau đó sẽ được quy đổi sang AQI ở lớp giao diện (trong app.py) để hiển thị cho người dùng.

(4) Chia dữ liệu train/test

Không dùng xáo trộn ngẫu nhiên (shuffle=False) để đảm bảo thứ tự thời gian.

```

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, shuffle=False
)

```

Dữ liệu được chia thành 80% cho train và 20% cho test bằng hàm train_test_split().

Để đảm bảo tính liên tục theo thời gian, tham số shuffle=False được dùng – nghĩa là dữ liệu không bị xáo trộn ngẫu nhiên. Cách này giúp mô hình học đúng xu hướng thời gian thực, phù hợp với bài toán dự đoán chuỗi thời gian PM2.5 (thay vì AQI).

(5) Huấn luyện mô hình LightGBM

Mô hình LightGBM Regressor được chọn vì khả năng xử lý dữ liệu lớn và tốc độ huấn luyện nhanh.

- Các tham số được thiết lập gồm:

- + objective='regression': bài toán dự báo giá trị liên tục (PM2.5).
- + metric='mae': dùng sai số tuyệt đối trung bình để đánh giá.
- + boosting_type='gbdt': thuật toán boosting dạng gradient tree.
- + learning_rate=0.05: tốc độ học vừa phải để tránh quá khớp.
- + num_leaves=31: số lượng lá trên cây quyết định độ phức tạp mô hình.
- + feature_fraction và bagging_fraction: giúp giảm overfitting bằng cách chỉ chọn ngẫu nhiên một phần đặc trưng và dữ liệu mỗi lần huấn luyện.

```
92     params = {
93         "objective": "regression",
94         "metric": "mae",
95         "boosting_type": "gbdt",
96         "learning_rate": 0.05,
97         "num_leaves": 31,
98         "feature_fraction": 0.8,
99         "bagging_fraction": 0.8,
100        "bagging_freq": 5,
101        "seed": 42,
102        "verbose": -1
103    }
```

- Mô hình được huấn luyện với early stopping để tránh overfitting:

```
108     model = lgb.train(
109         params=params,
110         train_set=train_data,
111         valid_sets=[val_data],
112         num_boost_round=2000,
113         callbacks=[lgb.early_stopping(stopping_rounds=100, verbose=False)]
114     )
```

Mô hình được huấn luyện với early stopping, nghĩa là nếu sai số trên tập kiểm định không giảm sau 100 vòng lặp, quá trình huấn luyện sẽ dừng lại sớm.

Điều này giúp mô hình không học quá kỹ dữ liệu huấn luyện (overfitting) và tổng quát tốt hơn khi dự đoán các giá trị PM2.5 (thay vì AQI) trong tương lai.

(6) Đánh giá mô hình

- ❖ Trường hợp 1: Không đủ dữ liệu để huấn luyện

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\aircare> .\venv312\Scripts\Activate.ps1
(.venv312) PS C:\Users\admin\aircare> python src/models/train_model.py
✓ Nạp dữ liệu: 24 dòng, 2025-11-12 21:45:45+07:00 → 2025-11-12 22:32:13+07:00

🚀 Bắt đầu huấn luyện mô hình PM2.5 (1h, 3h, 6h)...

✗ Không đủ dữ liệu (cần > 30 dòng) để huấn luyện cho 1h. Bỏ qua.
✗ Không đủ dữ liệu (cần > 90 dòng) để huấn luyện cho 3h. Bỏ qua.
✗ Không đủ dữ liệu (cần > 180 dòng) để huấn luyện cho 6h. Bỏ qua.

=====
☑ HIỆU SUẤT CÁC MÔ HÌNH DỰ BÁO PM2.5
=====
Không có mô hình nào được huấn luyện thành công (có thể do thiếu dữ liệu).
=====
✓ Huấn luyện hoàn tất.
```

❖ Trường hợp 2: Khi đã nạp đủ dữ liệu vào mô hình để huấn luyện

Sau khi huấn luyện, mô hình được đánh giá trên tập test (20% dữ liệu mới nhất) bằng hai chỉ số chính: MAE và R² (R-squared).

+ MAE (Mean Absolute Error): Cho biết sai số trung bình của mô hình.

+ R² (Hệ số xác định): Đo lường mức độ mô hình giải thích được sự biến động của dữ liệu (từ 0 đến 1).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\admin\aircare> .\venv312\Scripts\Activate.ps1
(.venv312) PS C:\Users\admin\aircare> python src/models/train_model.py
✓ Nạp dữ liệu: 1000 dòng, 2025-11-12 21:45:45+07:00 → 2025-11-17 19:43:43+07:00

🚀 Bắt đầu huấn luyện mô hình PM2.5 (1h, 3h, 6h)...

✓ PM2.5 +1h → MAE: 8.831 | R²: -0.096 | 📁 pm2_5_model_1h.pkl
✓ PM2.5 +3h → MAE: 14.047 | R²: -0.933 | 📁 pm2_5_model_3h.pkl
✓ PM2.5 +6h → MAE: 9.153 | R²: 0.320 | 📁 pm2_5_model_6h.pkl

=====
☑ HIỆU SUẤT CÁC MÔ HÌNH DỰ BÁO PM2.5
=====
🕒 1h → MAE: 8.831 | R²: -0.096
🕒 3h → MAE: 14.047 | R²: -0.933
🕒 6h → MAE: 9.153 | R²: 0.320
=====
✓ Huấn luyện hoàn tất.
```

- Mô hình dự báo 1 giờ (pm2_5_model_1h.pkl)

+ MAE = 8.831: mức sai số trung bình thấp, phù hợp cho các dự báo tức thời.

+ R² = -0.096: mô hình chưa bắt kịp độ biến thiên thực tế của dữ liệu.

+ Nhận xét:

- Mặc dù sai số dự đoán khá nhỏ, nhưng R² âm cho thấy dữ liệu PM2.5 biến động phức tạp theo thời tiết và mô hình chưa nắm bắt được hết quy luật.
- Tuy nhiên, mức MAE thấp vẫn cho phép dùng trong cảnh báo nhanh.

- Mô hình dự báo 3 giờ (pm2_5_model_3h.pkl)

+ MAE = 14.047: sai số tăng do khoảng dự báo dài hơn.

+ $R^2 = -0.033$: hiệu suất mô hình thấp, gần như tương đương dự đoán theo trung bình.

+ Nhận xét:

- Dự báo tầm trung (3h) luôn khó hơn vì PM2.5 chịu ảnh hưởng mạnh bởi: thời tiết biến đổi nhanh, hoạt động giao thông, độ ẩm thay đổi theo giờ.
- Mô hình 3 giờ chưa ổn định: cần nhiều dữ liệu hơn để học được xu hướng.

- Mô hình dự báo 6 giờ (pm2_5_model_6h.pkl)

+ MAE = 9.153: tốt hơn mô hình 3 giờ, sai số chấp nhận được.

+ $R^2 = 0.320$: đây là mô hình có chất lượng cao nhất trong 3 mô hình.

+ Nhận xét:

- Mô hình 6h học được xu hướng dài hạn tốt hơn.
- R^2 dương và đạt 0.32 cho thấy mô hình đã nắm bắt một phần mối quan hệ giữa dữ liệu đầu vào và PM2.5 tương lai.
- Đây là mô hình ổn định nhất nhờ xu hướng PM2.5 trong dài hạn ít nhiễu hơn.

Kết luận:

+ Mô hình 1h: Sai số thấp, thích hợp cho cảnh báo nhanh nhưng R^2 thấp, phù hợp cho cảnh báo chất lượng không khí theo thời điểm thực.

+ Mô hình 3h: Hiệu suất kém nhất do nhiễu trong dữ liệu thời gian ngắn.

+ Mô hình 6h: Là mô hình tốt nhất – R^2 cao nhất và sai số khá ổn định, được ưu tiên sử dụng cho các biểu đồ dự báo xu hướng.

→ Cần bổ sung thêm dữ liệu (tối thiểu 30–60 ngày) để cải thiện R^2 của mô hình 1h và 3h.

(7) Lưu mô hình

```
116 | # Đánh giá
117 | y_pred = model.predict(X_test)
118 | mae = mean_absolute_error(y_test, y_pred)
119 | r2 = r2_score(y_test, y_pred)
120 |
121 | model_path = os.path.join(MODEL_DIR, f"pm2_5_model_{horizon_in_hours}h.pkl")
122 | features_path = os.path.join(MODEL_DIR, f"pm2_5_features_{horizon_in_hours}h.pkl")
123 | joblib.dump(model, model_path)
124 | joblib.dump(features, features_path)
125 |
126 | print(f"✅ PM2.5 +{horizon_in_hours}h → MAE: {mae:.3f} | R²: {r2:.3f} | 📁 {os.path.basename(model_path)}")
127 | return {"horizon": horizon_in_hours, "mae": mae, "r2": r2}
128 |
```

Sau khi huấn luyện, mô hình LightGBM và danh sách đặc trưng được lưu lại bằng thư viện joblib để phục vụ cho việc dự đoán sau này:

+ pm2_5_model_{horizon}h.pkl: Chứa mô hình đã huấn luyện để dự đoán PM2.5 cho từng mốc dự báo (1h, 3h, 6h).

+ pm2_5_features_{horizon}h.pkl: Chứa danh sách các đặc trưng đầu vào tương ứng.

Việc lưu song song mô hình và đặc trưng giúp đảm bảo khi tải lại để dự đoán (trong app.py), dữ liệu đầu vào có cấu trúc thống nhất với giai đoạn huấn luyện.

4.5 Quy đổi PM2.5 sang AQI

Một điểm mấu chốt trong kiến trúc của dự án là sự phân tách rõ ràng giữa dự đoán (backend) và hiển thị (frontend):

- Lớp Huấn luyện (Backend): File src/models/train_model.py chỉ tập trung vào một nhiệm vụ duy nhất: dự đoán chính xác nồng độ PM2.5 ($\mu\text{g}/\text{m}^3$).

- Lớp Ứng dụng (Frontend): Người dùng cuối thường không hiểu ý nghĩa của "55.7 $\mu\text{g}/\text{m}^3$ ". Họ quen thuộc với thang đo AQI (0-500) hơn.

Vì vậy, toàn bộ logic quy đổi được xử lý tại lớp ứng dụng, cụ thể là trong file src/run/app.py

```
30 # =====
31 # ◆ HÀM QUY ĐỔI PM2.5 → AQI
32 # =====
33 def pm25_to_aqi(pm25):
34     """Quy đổi giá trị PM2.5 sang chỉ số AQI (chuẩn US EPA)"""
35     breakpoints = [
36         (0.0, 12.0, 0, 50, "Tốt", "Không khí trong lành – rất tốt cho sức khỏe."),
37         (12.1, 35.4, 51, 100, "Trung bình", "Người nhạy cảm có thể bị ảnh hưởng nhẹ."),
38         (35.5, 55.4, 101, 150, "Kém", "Người già, trẻ nhỏ, người bệnh nên hạn chế ra ngoài."),
39         (55.5, 150.4, 151, 200, "Xấu", "Ô nhiễm; tránh ra ngoài và hoạt động mạnh."),
40         (150.5, 250.4, 201, 300, "Rất xấu", "Rất ô nhiễm; ở trong nhà nếu có thể."),
41         (250.5, 500.0, 301, 500, "Nguy hại", "Không ra ngoài, nguy hiểm đến sức khỏe."),
42     ]
43     # Xử lý trường hợp PM2.5 rất cao
44     if pm25 > 500.0:
45         return 500, "Nguy hại", "Không ra ngoài, nguy hiểm đến sức khỏe."
46
47     for low, high, aqi_low, aqi_high, cat, msg in breakpoints:
48         if low <= pm25 <= high:
49             aqi = ((aqi_high - aqi_low) / (high - low)) * (pm25 - low) + aqi_low
50             return round(aqi, 1), cat, msg
51
52     # Trường hợp dự phòng nếu pm25 là âm hoặc NaN
```

Hàm này sử dụng Bảng các ngưỡng (Breakpoints) theo tiêu chuẩn của Cơ quan Bảo vệ Môi trường Hoa Kỳ (US EPA). Nó áp dụng công thức nội suy tuyến tính (linear interpolation) để tính toán chính xác giá trị AQI (0-500) từ bất kỳ giá trị PM2.5 nào (cả giá trị hiện tại và giá trị dự đoán từ mô hình).

5. Giao diện AIRCARE

5.1 Triển khai giao diện người dùng (UI) bằng Streamlit

Hệ thống AirCare sử dụng Streamlit để xây dựng giao diện người dùng trực quan, hỗ trợ người dùng tương tác với các chức năng như dự đoán AQI (từ pm2.5), xem biểu đồ, theo dõi sức khỏe và chatbot hỗ trợ.

```
○ (.venv312) PS C:\Users\admin\aircare> streamlit run src/run/app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.117:8501
```


Điều này giúp AirCare có thể chạy dưới dạng ứng dụng web nội bộ, không cần triển khai server công khai nhưng vẫn hỗ trợ nhiều thiết bị truy cập.

Ý nghĩa trong hệ thống

- + Streamlit giúp nhóm không cần xây dựng front-end phức tạp (React, HTML/CSS) nhưng vẫn tạo được giao diện đẹp và dễ sử dụng.
- + Tất cả các biểu đồ (Forecast AQI, lịch sử dữ liệu), chatbot, và mục nhập dữ liệu người dùng được hiển thị trực tiếp trên giao diện Streamlit.
- + Ứng dụng tự động cập nhật theo thay đổi của dữ liệu và model.

5.2. Chức năng chính

- Dự báo đa thời điểm (PM2.5): Sử dụng mô hình LightGBM đã huấn luyện để dự đoán nồng độ PM2.5 ($\mu\text{g}/\text{m}^3$) tại 3 mốc thời gian tương lai: 1 giờ, 3 giờ, và 6 giờ tới.
- Quy đổi PM2.5 sang AQI: Tự động quy đổi các giá trị PM2.5 (cả hiện tại và dự đoán) sang thang đo AQI (0-500) theo chuẩn US EPA, giúp người dùng dễ dàng hiểu mức độ ô nhiễm.
- Chatbot (AI Ngữ nghĩa): Nhận lệnh bằng ngôn ngữ tự nhiên (ví dụ: “Dự đoán AQI”, “Tôi nên ra ngoài không?”) và sử dụng mô hình SentenceTransformer để phát hiện chính xác ý định (intent) của người dùng.
- Cảnh báo sức khỏe cá nhân hóa: Cung cấp các khuyến nghị sức khỏe (ví dụ: “nên hạn chế ra ngoài”) dựa trên mức AQI dự đoán và thông tin bệnh lý (ví dụ: “hen suyễn”) do người dùng nhập vào.
- Biểu đồ dự báo trực quan: Minh họa biến động của AQI (đã quy đổi), bao gồm dữ liệu lịch sử gần nhất và dữ liệu dự báo cho 6 giờ tới.
- Thu thập dữ liệu tự động: Script collector.py (khi được kích hoạt) sẽ chạy nền và tự động gọi API OpenWeather để thu thập dữ liệu mới mỗi 2 phút.



Hình 5.1 Giao diện chính AIRCARE

5.3. Hiển thị thông tin cụ thể tại thời điểm hiện tại và đưa ra cảnh báo



5.4. Dự đoán AQI trong 1, 3, 6 giờ tới

- Nếu người dùng muốn xem biểu đồ AQI dự đoán



- Nếu người dùng muốn xem dữ liệu AQI được dự đoán



5.5. Cảnh báo khi người dùng nhập thông tin bệnh lý

 **Thông tin người dùng**

Tuổi

25 - +

Bệnh lý (nếu có)

xoang mũi

 Dự đoán nhanh

 **AIRCARE — Dự đoán AQI (từ PM2.5)** 

Hỏi về chất lượng không khí hoặc dự đoán AQI (từ PM2.5) trong 1h, 3h, 6h tới.

 **Chatbot**

Bạn: hi

Bot: Chào ! Tôi là AirCare Chatbot, sẵn sàng giúp bạn theo dõi chất lượng không khí .

Bạn: người bị xoang mũi có nên ra đường không

Bot: AQI hiện tại 166 — Xấu. Ô nhiễm; tránh ra ngoài và hoạt động mạnh. Vì bạn có bệnh 'xoang mũi', nên đặc biệt chú ý khi AQI > 100.

Nhập tin nhắn...

 **Dự báo & Cảnh báo**

 2025-11-17 21:02:35+07:00 — AQI hiện tại: 166 (Xấu)

Ô nhiễm; tránh ra ngoài và hoạt động mạnh.

AQI: 167.4 — Xấu

Ô nhiễm; tránh ra ngoài và hoạt động mạnh.

AQI: 161.9 — Xấu

Ô nhiễm; tránh ra ngoài và hoạt động mạnh.

AQI: 166.2 — Xấu

Ô nhiễm; tránh ra ngoài và hoạt động mạnh.

6. Hướng phát triển

Dự án AIRCARE có tiềm năng mở rộng lớn, hướng tới việc tích hợp sâu hơn vào hệ sinh thái Smart City và công nghệ AI tiên tiến.

Mở rộng phạm vi địa lý: Hiện tại hệ thống đang tập trung vào Hà Nội. Trong tương lai, dự án sẽ mở rộng khả năng thu thập dữ liệu và huấn luyện mô hình riêng biệt cho nhiều thành phố lớn khác như TP. Hồ Chí Minh, Đà Nẵng, Hải Phòng, cung cấp dự báo cục bộ chính xác hơn.

Tích hợp IoT (Internet of Things): Thay vì chỉ phụ thuộc vào API OpenWeather, hệ thống sẽ kết nối trực tiếp với các trạm cảm biến đo bụi thực tế (IoT sensors) đặt tại các điểm nóng ô nhiễm. Điều này giúp tăng độ tin cậy và giảm độ trễ của dữ liệu đầu vào.

Triển khai trên nền tảng đám mây (Cloud Deployment): Để phục vụ lượng người dùng lớn và đảm bảo tính sẵn sàng cao, hệ thống sẽ được chuyển từ môi trường cục bộ lên các nền tảng đám mây chuyên nghiệp như AWS SageMaker (để huấn luyện và phục vụ mô hình quy mô lớn) hoặc Streamlit Cloud (để triển khai giao diện người dùng nhanh chóng).

Nâng cấp trí tuệ Chatbot: Kết nối chatbot hiện tại (dựa trên SentenceTransformer) với các Mô hình Ngôn ngữ Lớn (LLM) mạnh mẽ hơn như GPT-4 hoặc Claude thông qua API. Điều này sẽ giúp chatbot không chỉ hiểu ý định đơn giản mà còn có khả năng giải thích chi tiết các hiện tượng thời tiết, tư vấn sức khỏe chuyên sâu và tương tác tự nhiên hơn với người dùng.

Phát triển ứng dụng di động (Mobile App): Xây dựng phiên bản ứng dụng di động đa nền tảng sử dụng Flutter, cho phép người dùng nhận thông báo đẩy (push notifications) về chất lượng không khí xấu ngay lập tức trên điện thoại, thay vì phải truy cập vào giao diện web thụ động.

TÀI LIỆU THAM KHẢO

Mã nguồn dự án GitHub: https://github.com/agnesskt/HTTM_NHOM8

[1] OpenWeatherMap. *API Documentation*. Truy cập từ: <https://openweathermap.org/api>

[2] **World Health Organization (WHO)**. *Ambient (outdoor) air pollution*.
[https://www.who.int/news-room/fact-sheets/detail/ambient-\(outdoor\)-air-quality-and-health](https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-quality-and-health)

[3] Microsoft. *LightGBM Official Guide*. Truy cập từ: <https://lightgbm.readthedocs.io>

[4] Hugging Face. *SentenceTransformer — “paraphrase-multilingual-MiniLM-L12-v2”*.
<https://www.sbert.net>

[5] Altair Developers. *Altair Visualization Library*. Truy cập từ: <https://altair-viz.github.io>

[6] **United States Environmental Protection Agency (EPA)**. *Air Quality Index (AQI) Basics*.
<https://www.airnow.gov/aqi/aqi-basics/>

[7] **U.S. EPA Technical Assistance Document**. *Guidelines for Reporting of Daily Air Quality – the AQI*.
<https://www.airnow.gov/publications/air-quality-index/>

[8] **Streamlit Documentation**. *Build data apps in Python*.
<https://docs.streamlit.io/>

[9] **Zhang, X., et al.** (2020). *Air pollution prediction using machine learning models: A review*.
Environmental Science and Pollution Research.

[10] **Wang, P., et al.** (2019). *PM2.5 Forecasting using Random Forest and Time Series Models*.
Atmospheric Pollution Research.

[11] **Li, T., et al.** (2017). *An integrated data preprocessing approach for air quality prediction*.
IEEE Access.

[12] **Pang, N., et al.** (2021). *Improving PM2.5 Prediction by Feature Engineering and Weather Data Fusion*.
Environmental Modelling & Software.